CCA 2 – Student Guide

CI/CD Pipeline Implementation and Deployment of an Existing Project TYCSE · Cloud Computing and DevOps · **Total Marks: 10**

1. What You Have to Do

Take a software project you built earlier (individually or in a team, this semester or a previous one) and build an **automated CI/CD pipeline** for it. You do **not** need to build a new project. Your marks depend on how well you automate delivery, not on how complex the application is.

Your pipeline must do all of this automatically through Jenkins:

1. Pull the latest code from GitHub.
2. Build the application and run available tests.
3. Create a Docker image.
4. Push the image to Docker Hub or AWS ECR.
5. Deploy the application on Minikube using Kubernetes manifests.
6. Let you verify that the application is running.

After the first successful run, you must **change the source code, push it, and run the pipeline again** to show the new version being deployed.

2. Tools You Will Use

Tool	What you use it for
GitHub	Stores your project code, Dockerfile, Jenkinsfile and Kubernetes files
AWS	Hosts the server where Jenkins (and Docker/Minikube) run
Jenkins	Runs your pipeline automatically using a Jenkinsfile
Docker	Packages your project into an image
Docker Hub / AWS ECR	Stores the image so Kubernetes can pull it
Minikube and kubectl	Run a local Kubernetes cluster and let you manage and check it

3. Pipeline Stages

Step	Stage	What must happen
1	Source Code	Project is available in a GitHub repository
2	Jenkins Checkout	Jenkins pulls the latest code
3	Build and Test	Application is built and tests are run
4	Docker Build	Docker image is created from your Dockerfile
5	Push Image	Image is published to Docker Hub or AWS ECR
6	Minikube Deployment	Kubernetes Deployment and Service are applied
7	Verification	Pods, deployment and service checked; app opened

Tip: If your project has no tests, use a build, lint or syntax-check step as your test stage and be ready to explain it.

4. Mandatory Requirements

1. Use an existing academic project.
2. Keep the source code in a GitHub repository.
3. Configure Jenkins to run the pipeline using a **Jenkinsfile**.
4. Create a **Dockerfile** for the project.
5. Build and publish the Docker image to Docker Hub or Amazon ECR.
6. Create Kubernetes **Deployment** and **Service** YAML files.
7. Deploy the application on Minikube.
8. Verify running pods, deployment, service and application accessibility.
9. Demonstrate a source-code change and run the pipeline again.
10. Submit all items listed in Section 5.

5. What to Submit

- GitHub repository URL
- Jenkinsfile
- Dockerfile
- Kubernetes Deployment and Service manifests
- Screenshots
- A brief README or report explaining your deployment process

Your repository should contain, at minimum: the project source, `Dockerfile`, `Jenkinsfile`, and a `k8s/` folder with the Deployment and Service files.

Your README/report should briefly cover: the project you chose, the pipeline stages and what each does, how you set up Jenkins on AWS, how you pushed the image, how you deployed and accessed the app, and what you changed for the second run.

6. Evaluation Rubric (10 Marks)

Assessment criteria	Marks
Existing project selection, GitHub repository and source-code readiness	1
AWS infrastructure and Jenkins configuration	1.5
Dockerfile and successful image build	1.5
Jenkinsfile and automated build/test pipeline	2
Container registry integration and image push	1
Kubernetes Deployment and Service on Minikube	1.5
Application verification, demonstration and viva	1.5
Total	**10**

What evaluators expect for each criterion

Criterion	Full marks look like
Project and GitHub (1)	A real academic project in a clean, well-organised repository that builds without manual fixes
AWS and Jenkins (1.5)	Jenkins running on AWS, accessible, with required plugins and credentials configured and a pipeline job connected to your repository
Dockerfile (1.5)	A working Dockerfile that builds successfully and produces a container that runs the application
Jenkinsfile (2)	A declarative pipeline with clear stages for checkout, build/test, Docker build, push and deploy that runs automatically without manual steps
Registry (1)	Image pushed from Jenkins to Docker Hub or ECR with a meaningful tag, using credentials stored securely in Jenkins

Criterion	Full marks look like
Kubernetes (1.5)	Valid Deployment and Service files; pods running on Minikube and pulling your image from the registry
Verification and viva (1.5)	Live demo of the running app, a code change flowing through the pipeline, and clear answers to questions

Performance expectations

- **Excellent (9–10):** Pipeline runs successfully, the application is accessible, you explain every stage and demonstrate a code-change redeployment.
- **Good (7–8):** Most stages work, with minor issues that you can troubleshoot.
- **Satisfactory (5–6):** Containerization and deployment work, but pipeline automation is incomplete.
- **Needs improvement (below 5):** You cannot demonstrate a working deployment or explain your configuration.

Questions? Contact **Priya Nagargoje, 9689714646 priya.nagargoje@mitwpu.edu.in**

Priya Nagargoje

Course Coordinator